



Big Picture

LLMs make next-word predictions in 3 main steps:

1. Turn words into numbers.
2. Mix those numbers using simple formulas (mostly weighted sums).
3. Turn the final scores into probabilities for each possible next word.

Everything is built from ideas they already know: vectors, weighted averages, and exponentials.

1. Words as Vectors (Lists of Numbers)

- Each word (or sub-word) is stored as a **vector**:
 - Example (tiny toy version):
 - "cat" → [1.2, -0.3, 0.7]
 - "dog" → [1.1, -0.2, 0.6]
- You can say: "The model doesn't see 'cat' as letters c-a-t, it sees it as a list of numbers that capture its meaning."

Mathematically, a vector is just:

$$\mathbf{x} = [x_1, x_2, \dots, x_d]$$

2. Weighted Sums (Matrix Multiplication Idea)

The core operation everywhere is:

New numbers = weights × old numbers

- If we have a vector $\mathbf{x} = [x_1, x_2]$ and a 2×2 matrix of weights: $W = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$
- Then the new vector $\mathbf{y} = W\mathbf{x}$ is: $y_1 = ax_1 + bx_2$, $y_2 = cx_1 + dx_2$

So each new number is just a **weighted sum** of the old numbers. The whole network is basically lots of these weighted sums stacked together.

You can say: "Every layer is just: multiply by some weights, add things up, then apply a simple non-linear rule."

3. Attention: "How Much Should I Look at Each Previous Word?"

Attention is the special trick in transformers.

Idea (no heavy notation):

- For the current word position, the model:
 1. Looks at all previous word vectors.
 2. Computes a **score** for each previous word: "How relevant is this word to me?"
 3. Turns those scores into **weights** that add up to 1 (like percentages).
 4. Takes a **weighted average** of the previous word vectors using those weights.

So if we have 3 previous words with vectors (v_1, v_2, v_3) and attention weights (w_1, w_2, w_3) (where $(w_1 + w_2 + w_3 = 1)$), the attention output is:

$$[\text{output} = w_1v_1 + w_2v_2 + w_3v_3]$$

That's just a **weighted average** of vectors.

You can phrase it as: "The model decides how much to pay attention to each earlier word, then averages them with those weights."

4. Nonlinear "Bumps" (Activation Functions)

If we only did weighted sums, the model would be too simple. So after some of these sums, we apply a simple rule to each number, like:

- **ReLU**: $[\text{ReLU}(x) = \max(0, x)]$
 - If x is negative $\rightarrow$ make it 0
 - If x is positive $\rightarrow$ keep it

This is just: “cut off all negative values.” It lets the model draw more complex shapes than straight lines.

5. From Scores to Probabilities: Softmax

At the very end, the model has one **score** for each possible next word in the vocabulary.

Call these scores: $[z_1, z_2, \dots, z_V]$ (one for each of V words).

We want to turn these into **probabilities** that:

- are all positive
- add up to 1

The function that does this is **softmax**:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}$$

You don’t need calculus here—just:

- (e^{z_i}) means “raise e to the power of (z_i) ” (like (2^{z_i}) , but with base e).
- Bigger scores $(z_i) \rightarrow$ much bigger $(e^{z_i}) \rightarrow$ higher probability.
- Dividing by the sum makes all probabilities add up to 1.

You can say: “Softmax takes raw scores and turns them into a nice probability distribution over all possible next words.”

6. How the Next Word Is Chosen

Once we have probabilities for each word:

- **Greedy**: pick the word with the highest probability.
- **Sampling**: randomly pick a word, but more likely if its probability is higher (like drawing from a weighted lottery).

That’s literally how the model “decides” the next word.